

UniApp iOS原生插件开发 极简讲解

UniApp iOS原生插件开发 极简讲解

UniApp的iOS原生插件，核心是通过Objective-C/Swift编写原生功能模块，封装成UniApp可调用的接口，弥补UniApp跨端API的不足（比如调用iOS独有系统能力、硬件交互等），以下是核心流程和关键点：

一、开发前准备

1. **环境要求**：Mac电脑 + Xcode（建议最新稳定版） + HBuilderX（或CLI工具）；
2. **核心依赖**：UniApp提供的 `UniPlugin.h` 头文件（封装了UniApp与原生的通信桥接），需提前引入到Xcode工程；
3. **开发语言**：优先Objective-C（UniApp插件桥接对OC支持更完善），Swift需通过桥接文件适配。

二、核心开发步骤（最简版）

1. 创建Xcode插件工程

- 新建“Framework”类型工程（iOS -> Framework），命名如 `UniCustomPlugin`；
- 引入UniApp插件核心文件：将 `UniPlugin.h`（可从UniApp官方插件示例获取）添加到工程，设置工程的 `iOS Deployment Target`（建议 ≥ 11.0 ，适配多数设备）。

2. 编写原生插件逻辑

新建OC类（如 `CustomPlugin`），实现UniApp插件协议 `UniModuleProtocol`，核心是暴露“供UniApp调用的方法”：

Code block

```
1  #import <Foundation/Foundation.h>
2  #import "UniPlugin.h"
3
4  @interface CustomPlugin : NSObject <UniModuleProtocol>
5  // 暴露给UniApp的方法（必须加UNI_EXPORT_METHOD宏）
6  UNI_EXPORT_METHOD(@selector(testiOSFunc:callback:))
7  - (void)testiOSFunc:(NSDictionary *)params callback:
    (UniModuleCallback)callback;
8  @end
9
10 @implementation CustomPlugin
```

```

11 - (void)testiOSFunc:(NSDictionary *)params callback:
    (UniModuleCallback)callback {
12     // 1. 接收UniApp传入的参数
13     NSString *msg = params[@"msg"];
14
15     // 2. 执行iOS原生逻辑（比如调用相册、推送、系统弹窗等）
16     NSString *result = [NSString stringWithFormat:@"iOS原生处理: %@", msg];
17
18     // 3. 回调结果给UniApp（成功/失败）
19     if (callback) {
20         // 成功回调：第一个参数@YES，第二个参数是返回数据
21         callback(@YES, @{@"code": @0, @"data": result});
22         // 失败回调示例：callback(@NO, @{@"code": @-1, @"msg": @"处理失败"});
23     }
24 }
25 @end

```

- 关键： `UNI_EXPORT_METHOD` 宏是核心——标记该方法可被UniApp调用；
 `UniModuleCallback` 是回调闭包，用于向UniApp返回结果。

3. 注册插件

新建 `module.json` 配置文件（放在插件工程根目录），声明插件信息，让UniApp识别：

Code block

```

1  {
2      "plugins": [
3          {
4              "name": "CustomPlugin", // 插件名（UniApp调用时用）
5              "class": "CustomPlugin" // 对应OC类名
6          }
7      ]
8  }

```

4. 编译插件

- 在Xcode中选择“Generic iOS Device”或指定真机，编译生成 `.framework` 文件；
- 整理插件包：将编译后的 `UniCustomPlugin.framework` + `module.json` 打包成 `CustomPlugin.zip`（目录结构需规范：根目录含 `ios` 文件夹，里面放framework，同级放module.json）。

5. UniApp中调用插件

- 在HBuilderX中，将插件包导入UniApp项目（manifest.json -> App原生插件配置 -> 本地插件引入）；
- 在JS代码中调用：

Code block

```
1  // 获取插件实例
2  const customPlugin = uni.requireNativePlugin('CustomPlugin');
3  // 调用原生方法
4  customPlugin.testiOSFunc({msg: 'Hello iOS原生'}, (res) => {
5    if (res.code === 0) {
6      console.log('成功: ', res.data);
7    } else {
8      console.log('失败: ', res.msg);
9    }
10  });
```

三、关键注意点

1. **通信规则**：UniApp与iOS原生之间仅支持JSON格式数据传输（对象、数组、基本类型），无法直接传递复杂对象；
2. **线程问题**：原生插件方法默认在子线程执行，若操作UI（如弹窗），需切到主线程：

Code block

```
1  dispatch_async(dispatch_get_main_queue(), ^{
2    // UI操作代码
3  });
```

3. **调试方式**：可将插件工程与UniApp打出来的iOS原生工程（HBuilderX -> 发行 -> 原生App-本地打包 -> 导出iOS工程）合并，直接在Xcode中调试；
4. **权限处理**：若调用相机、定位等敏感权限，需在 `Info.plist` 中添加权限描述（如 `NSCameraUsageDescription`）。

四、核心逻辑总结

iOS原生插件本质是“遵循UniApp协议的OC/Swift模块”，通过 `UNI_EXPORT_METHOD` 暴露方法，以 `module.json` 声明插件，最终在UniApp中通过 `requireNativePlugin` 调用，核心是做好“参数传递”和“回调结果”的适配，其余就是纯iOS原生开发的逻辑实现。

（注：文档部分内容可能由 AI 生成）